

62. Resumable File Upload

`fetch` se file upload aasan hai. Par connection toot jaye to upload **wahin se dobara** kaise shuru kare? Iske liye koi built-in option nahi, par hum khud bana sakte hain.

Bade files (jahan resume ki zarurat) me progress bhi chahiye, aur `fetch` upload progress nahi deta, isliye `XMLHttpRequest` use karenge.

Progress event kaafi kyun nahi?

Resume karne ke liye pata hona chahiye ki server ko **exactly kitne bytes mile**.

`xhr.upload.onprogress` sirf batata hai ki data **bheja** gaya, ye nahi ki server ko **mila** (beech me proxy me buffer ho sakta hai, server process mar sakta hai, ya packet raste me kho sakta hai).

Isliye ye event sirf achhe progress bar ke kaam ka hai. Sahi count sirf **server** bata sakta hai, to ek extra request chahiye.

Algorithm

1. File ki unique ID banao:

```
js
let fileId = file.name + '-' + file.size + '-' + file.lastModified;
```

Naam, size ya modified date badle to alag ID banegi.

2. Server se poochho kitne bytes mil chuke hain:

```
js
let response = await fetch('status', {
  headers: { 'X-File-Id': fileId }
});
let startByte = +await response.text(); // server ke paas itne bytes hain
```

Server `X-File-Id` header se upload track karta hai (server side implement karna hoga). File na ho to server `0` deta hai.

3. `Blob.slice` se `startByte` ke baad ka hissa bhejo:

```
js
```

```
xhr.open("POST", "upload");
xhr.setRequestHeader('X-File-Id', fileId);          // kaun si file
xhr.setRequestHeader('X-Start-Byte', startByte);    // kahan se resume
xhr.upload.onprogress = (e) => {
  console.log(`Uploaded ${startByte + e.loaded} of ${startByte + e.total}`);
};
xhr.send(file.slice(startByte));
```

Server ko `X-File-Id` se pata hota hai kaun si file hai, aur `X-Start-Byte` se ki ye resume hai. Server apne records dekhta hai: agar us file ka abhi ka uploaded size **exactly** `X-Start-Byte` hai to data usme **append** karta hai.

Client (Uploader class ka sar)

js

```

class Uploader {
  constructor({file, onProgress}) {
    this.file = file;
    this.onProgress = onProgress;
    this.fileId = file.name + '-' + file.size + '-' + file.lastModified;
  }

  async getUploadedBytes() {
    let response = await fetch('status', { headers: { 'X-File-Id': this.fileId } });
    if (response.status !== 200) throw new Error("Can't get uploaded bytes: " + response.status);
    return +(await response.text());
  }

  async upload() {
    this.startByte = await this.getUploadedBytes();

    let xhr = this.xhr = new XMLHttpRequest();
    xhr.open("POST", "upload", true);
    xhr.setRequestHeader('X-File-Id', this.fileId);
    xhr.setRequestHeader('X-Start-Byte', this.startByte);

    xhr.upload.onprogress = (e) => {
      this.onProgress(this.startByte + e.loaded, this.startByte + e.total);
    };

    xhr.send(this.file.slice(this.startByte));

    return await new Promise((resolve, reject) => {
      xhr.onload = xhr.onerror = () => {
        if (xhr.status === 200) resolve(true);
        else reject(new Error("Upload failed: " + xhr.statusText));
      };
      xhr.onabort = () => resolve(false); // sirf xhr.abort() par
    });
  }

  stop() {
    if (this.xhr) this.xhr.abort();
  }
}

```

- `upload()` (`true`) (success), (`false`) (abort) deta hai, error par throw.

Server (Node.js) ka idea

- `uploads[fileId]` me `bytesReceived` yaad rakho.
- `/status` request par bytes ki ginti (ya `0`) bhejo.
- `/upload` par: `startByte` `0` ho to nayi file, warna check karo `bytesReceived == startByte` (nahi to 400 "Wrong start byte"), phir file me **append**.
- Poori file aane par record hata do; connection toota to adhoori file chhod do.

Yaad rakho

Modern networking methods file managers jaisi taqat dete hain: headers par control, progress indicator, file ke hisse bhejna. Isse resumable upload aur bahut kuch banaya ja sakta hai.